Tecnológico de Antioquia
Prof Diego José Luis Botia
2026

Laboratorio 1 Ingeniería de Software II. Integración Java EE y ORM Hibernate

En este laboratorio utilizaremos el IDE Netbeans para crear y desplegar una aplicación Web que muestre datos desde una base de datos. La aplicación utiliza el framework Hibernate en la capa de persistencia para generar POJOs desde la base de datos relacional.

Hibernate es un framework que da herramientas para realizar operaciones ORM (object relational mapping). Este framework esta soportado por el IDE y presenta una serie de asistentes para crear los archivos Hibernate necesarios. Después de crear los objetos Java y configurar la aplicación para utilizar Hibernate, crearemos una interfaz gráfica de usuario web (GUI) para buscar y desplegar los datos.

Hibernate no solo realiza esta esta transformación sino que nos proporciona capacidades para la obtención y almacenamiento de datos de la base de datos que nos reducen el tiempo de desarrollo.

En este laboratorio se utilizará MySQL.

Para empezar este laboratorio se recomienda leer la siguiente página:

- Documentación Oficial de Hibernate www.hibernate.org.

Software Requerido.

Software o Recurso	Versión Requerida
NetBeans IDE	8.2 o superior
Java Development Kit (JDK)	11 o superior
Framework Hibernate	3.x o superior
Servidor de Aplicaciones Glassfish	4.x o superior
MySQL database server	versión 5.x o superior

Arquitectura

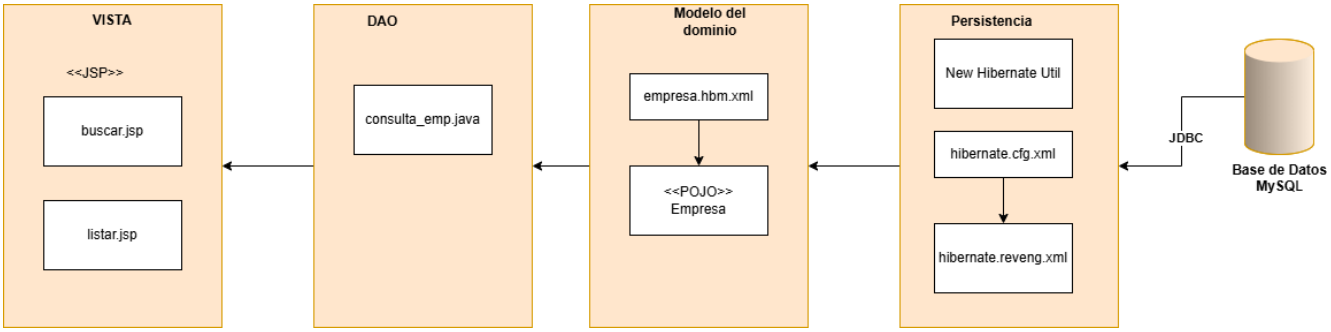


Figura 1 Arquitectura

Procedimiento.
1- Creación de la Base de datos

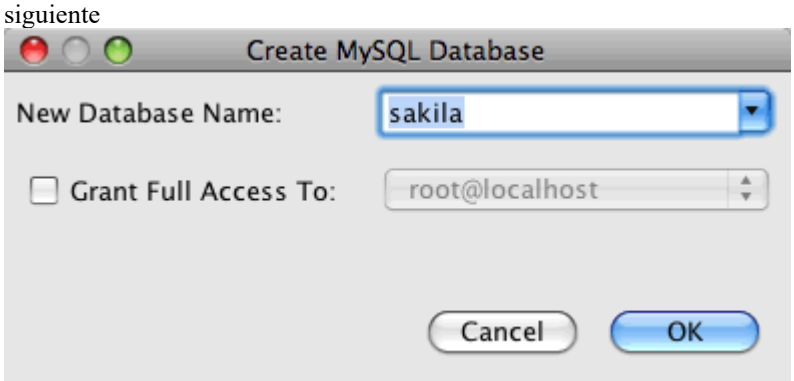
1. Crear una base de datos llamada Public
2. Crear una tabla llamada “empresa”

Nota: Solo se crea una tabla para mostrar el funcionamiento básico de Hibernate. Se recomienda hacer la práctica con tablas relacionadas.

3. La tabla tendrá los siguientes campos

rut (varchar 50)
nombre (Varchar 50)
direccion (Varchar 50)

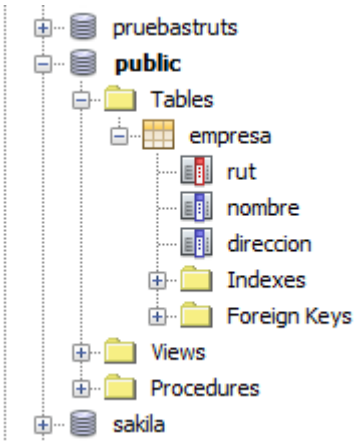
1. Expanda el nodo de base de datos en la ventana Services, haga click con el botón derecho del ratón sobre el nodo MySQL Server y elija Start.
2. Una vez inicie el servidor de MySQ haga click derecho de nuevo sobre el nodo y elija Create Database.
3. Escriba el nombre de la base de datos desde el combo box New Database Name y de click en OK como se ve en la siguiente figura.



Cuando haga click en OK el nodo empresa aparecerá debajo del nodo de MySQL Server.

4. Haga click derecho sobre el nodo empresa y seleccione Connect.

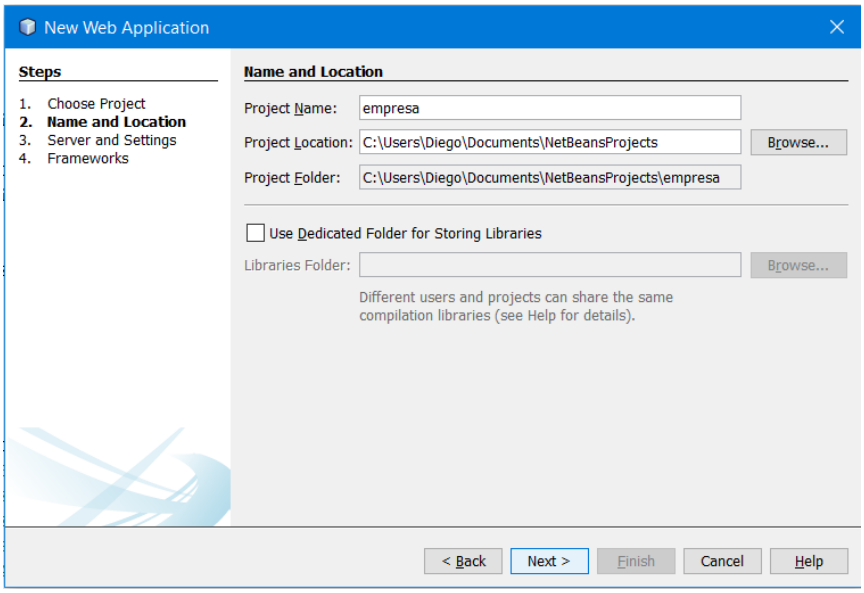
Ahora aparecerá el nodo de conexión a la base de datos mostrando lo siguiente: (`jdbc:mysql://localhost:3306/empresa [`*`username`*` on Default]`). Cuando la conexión es abierta usted puede ver los datos en cada tabla de la base de datos.



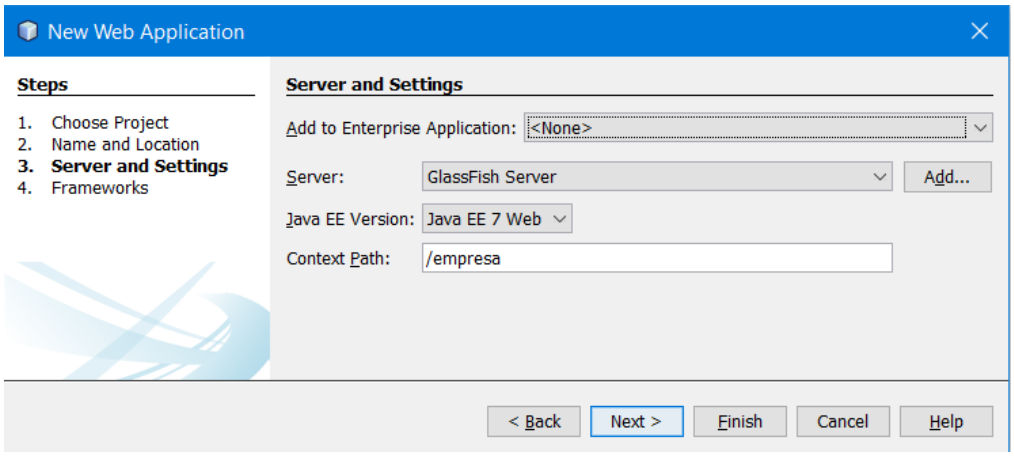
Creando un Java Web Application Project

A continuación, crearemos un proyecto web de tipo Java Web application llamado **empresa**

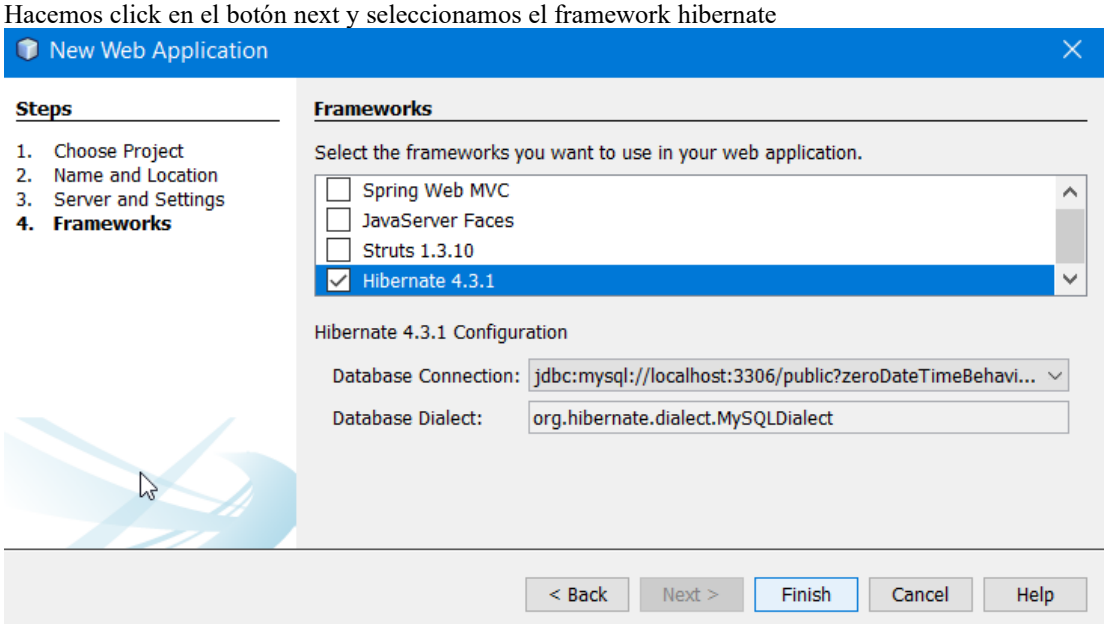
1. Seleccione File > New Project (Ctrl-Shift-N). Seleccione Web Application de la categoría Java Web y haga click en Next.
2. Escriba **Empresa** en el nombre del proyecto.
3. En la opción Use Dedicated Folder no la seleccione. Esto es porque no necesitaremos compartir las librerías con otros usuarios.



4. Seleccionamos el servidor de aplicaciones (Por ejeplo Galssfish) y en JavaEE Versión podemos seleccionar JavaEE 7 o superior.



5. Hacemos click en el botón next y seleccionamos el framework hibernate



6. Nos aseguramos que la BD esté conectada

Adicionando el soporte de Hibernate al proyecto

Para adicionar soporte a Hibernate desde un proyecto JavaEE usted necesita adicionar las librerías de Hibernate al proyecto. Las librerías de hibernate están incluidas en el IDE y pueden ser adicionadas manualmente o automáticamente (en el caso de utilizar asistentes).

El IDE incluye asistentes que ayudan a crear los archivos de Hibernate que usted puede necesitar en su proyecto. Usted puede crear así los archivos de configuración de Hibernate y las clases helper.

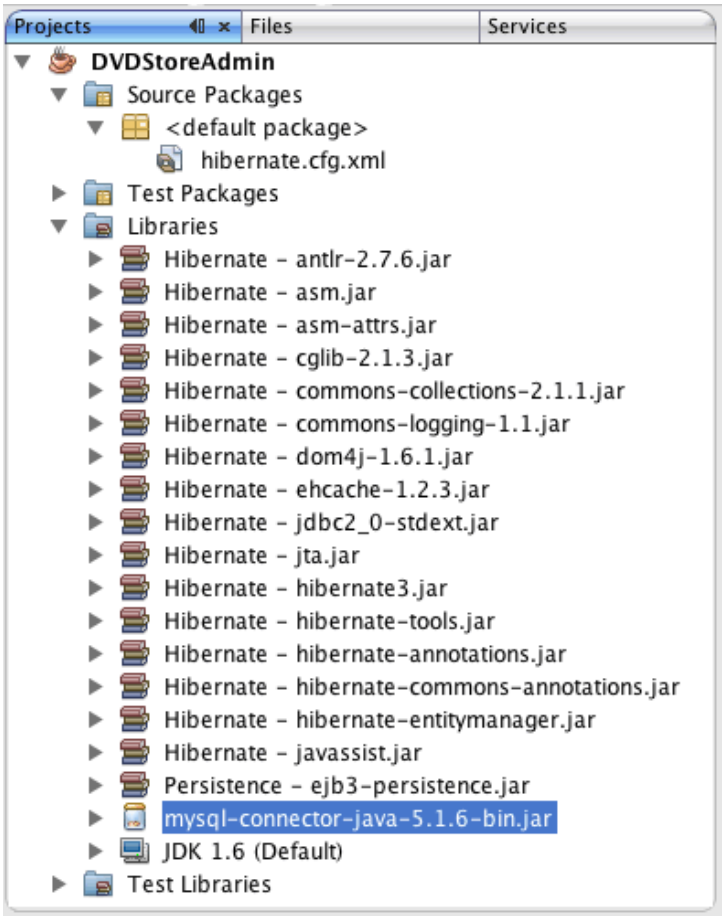
Creando el archivo de configuración de Hibernate

El archivo de configuración de Hibernate (`hibernate.cfg.xml`) contiene información sobre la conexión a la base de datos, recursos para mapeos, y otras propiedades de conexión. Cuando usted crea el archivo de configuración utilizando el asistente usted debe especificar la conexión a la base de datos seleccionándolo desde la lista de conexiones registradas en Netbeans. Cuando se genera el archivo de configuración el IDE automáticamente adiciona los detalles de la conexión y la información del dialecto basado en la conexión de BD seleccionado. El IDE adiciona además automáticamente las librerías

de Hibernate al classpath del proyecto. Después de crear el archivo de configuración usted puede editar el archivo utilizando el editor de múltiples vistas o editando directamente el archivo en un editor de XML.

El IDE crea un archivo de configuración en la raíz del classpath del contexto de la aplicación. En la ventana de proyectos el archivo es localizado en el source package `<default package>`. El archivo de configuración contiene información sobre una única base de datos. Si usted planea conectar a múltiples bases de datos, usted puede crear múltiples archivos de configuración en el proyecto, uno por cada servidor de base de datos.

Si usted expande el nodo en la ventana del proyecto usted podrá ver que el IDE agrego los archivos JAR de Hibernate y el archive JAR del driver de MySQL como se ve a continuación.

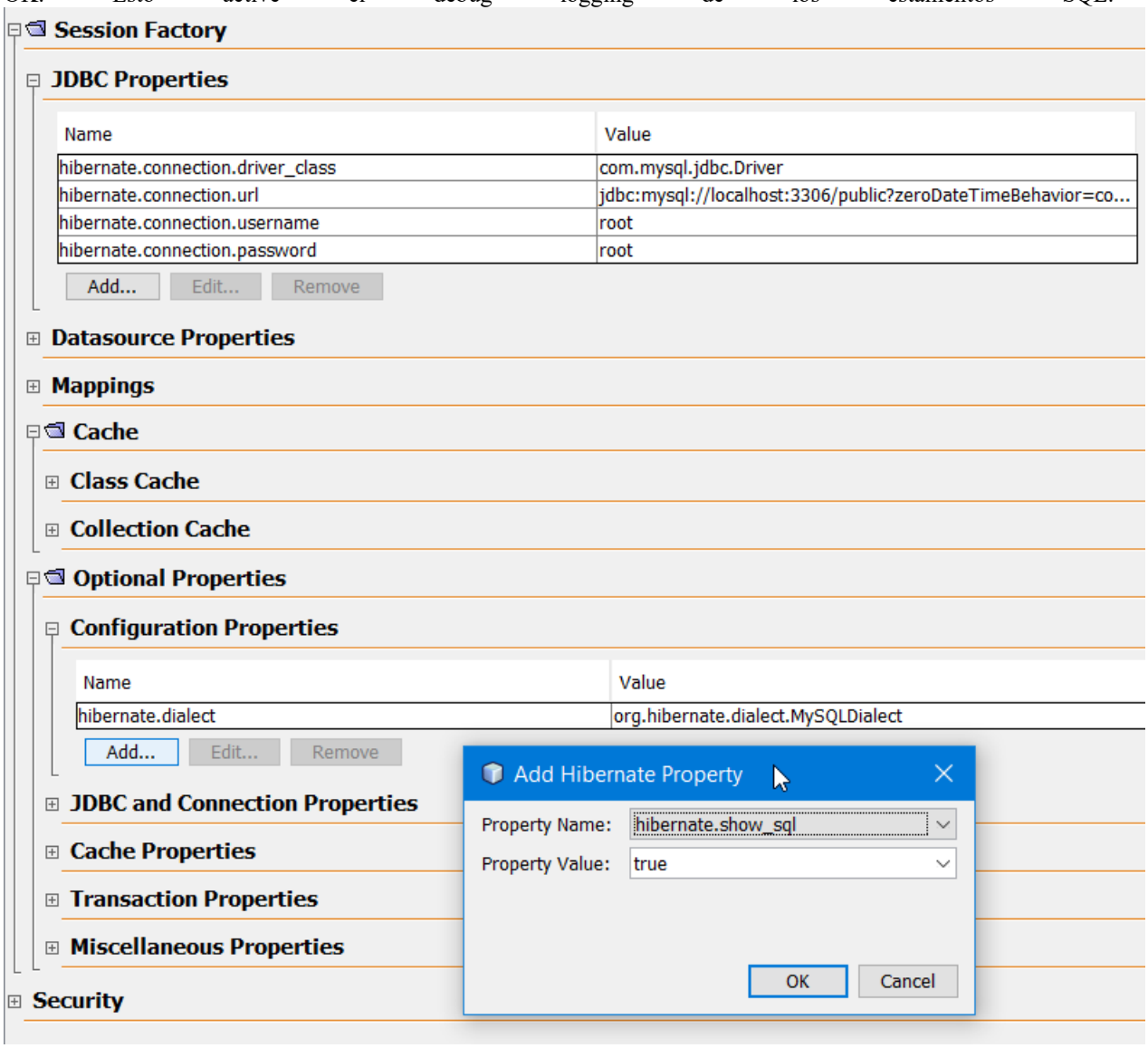


Modificando el archive de configuración de Hibernate

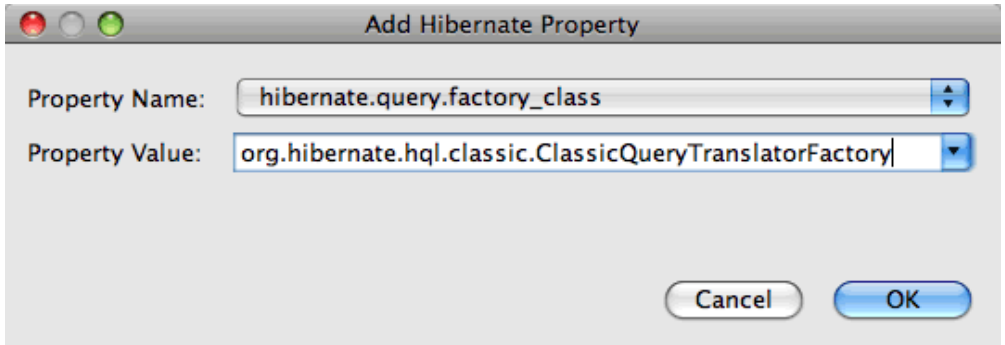
En esta parte usted deberá editar las propiedades por defecto especificadas en el archivo `hibernate.cfg.xml` para activar el debug logging para los estamentos SQL y activar las creación de sesiones desde el SessionFactory de Hibernate. .

1. Abra el archive `hibernate.cfg.xml` en el tabulador Design. Usted podrá abrir el archivo expandiendo en el nodo Configuration Files en la ventana del proyecto y hacienda double-click en `hibernate.cfg.xml`.
2. Expanda el nodo propiedades de configuración debajo de Optional Properties.
3. Haga click en Add para abrir el cuadro de dialogo Add Hibernate Property

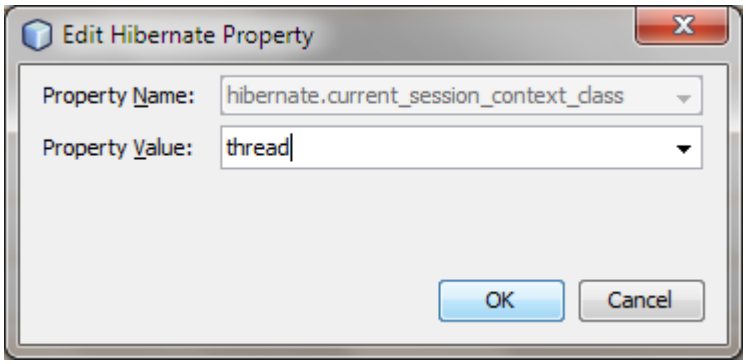
4. En el cuadro de dialogo seleccione la propiedad `hibernate.show_sql` y deje el valor en `true`. Haga click en OK. Esto active el debug logging de los estamentos SQL.



5. Haga click en Add debajo del nodo Miscellaneous Properties y seleccione `hibernate.query.factory_class` en el combo box Property Name
6. Seleccione **org.hibernate.hql.classic.ClassicQueryTranslatorFactory** como valor de la propiedad . Haga click en OK.



7. Seleccione de nuevo Add y elija en Property Name: hibernate.current_session_context_class y en el valor de la propiedad coloque thread



Si usted hace click en el separador XML en el editor usted vera la siguiente vista XML:

```
<hibernate-configuration>
    <session-factory name="session1">
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/empresa</propert
y>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">######</property>
        <property name="hibernate.show_sql">true</property>
        <property
name="hibernate.query.factory_class">org.hibernate.hql.classic.ClassicQueryT
ranslatorFactory</property>
    </session-factory>
</hibernate-configuration>
```

8. Guarde los cambios en su archivo.

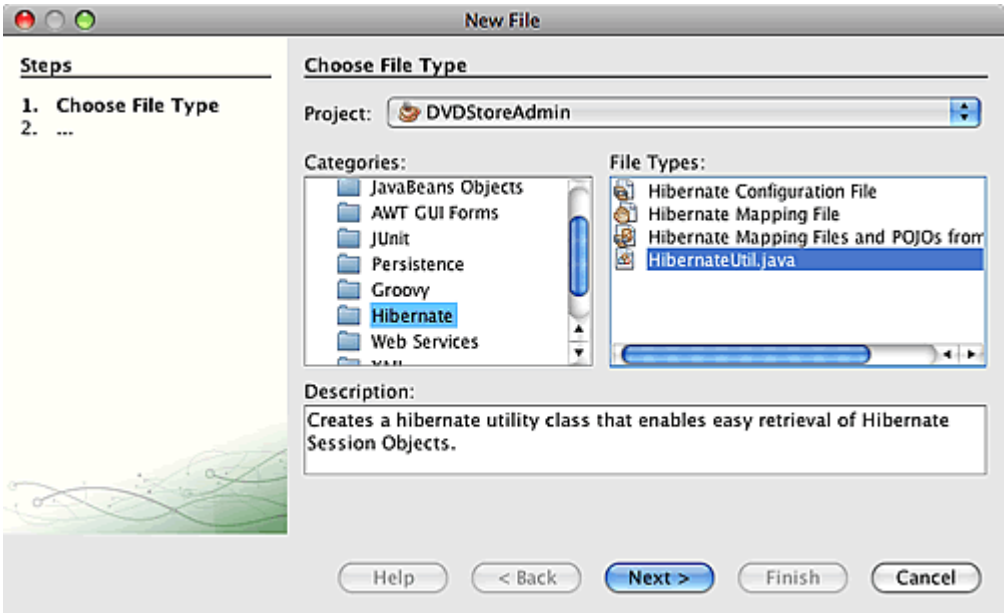
Después de que usted cree el formulario y configure la clase main, usted podrá ver las consultas SQL impresas en la ventana de salida del IDE cuando el proyecto se ejecute.

Creando el archivo Helper `HibernateUtil.java`

Para utilizar Hibernate usted necesita crear una clase helper que se toma desde el inicio y accede al **SessionFactory** para obtener el objeto de tipo **session**. La clase llama al método `configure()` de hibernate, carga el archivo de configuración `hibernate.cfg.xml` y luego construye el `SessionFactory` para obtener el objeto session .

Ahora se utilizará el asistente para crear la clase helper llamado `HibernateUtil.java`.

1. Haga click derecho sobre el nodo source packages y seleccione New > Other .
2. Seleccione Hibernate desde la lista de categorías y luego elija HibernateUtil.java. Haga click en Next.
3. Escriba **HibernateUtil** para el nombre de la clase y **session** para el nombre del paquete. Haca click en Finish.



Generando el archivo de Mapeo de Hibernate y las Clases POJO

Ahora utilizaremos un a **plain old Java object (POJO)**, denominado `Empresa.java`, para representar los datos en la tabla EMPRESA en la base de datos. La clase especifica los campos para las columnas en las tablas y utiliza simples métodos getters y setters para obtener o escribir datos. El mapeo de Empresa.java a la tabla EMPRESA usted puede usted puede utilizar el archivo de mapeo Hibernate o utilizar anotaciones en la clase. Usted puede usar el asistente de ingeniería inversa el asistente llamado **Hibernate Mapping Files and POJOs from a Database** para crear múltiples POJOs y archivos de mapeo basados en las tablas de la BD que usted selecciono.

Nota

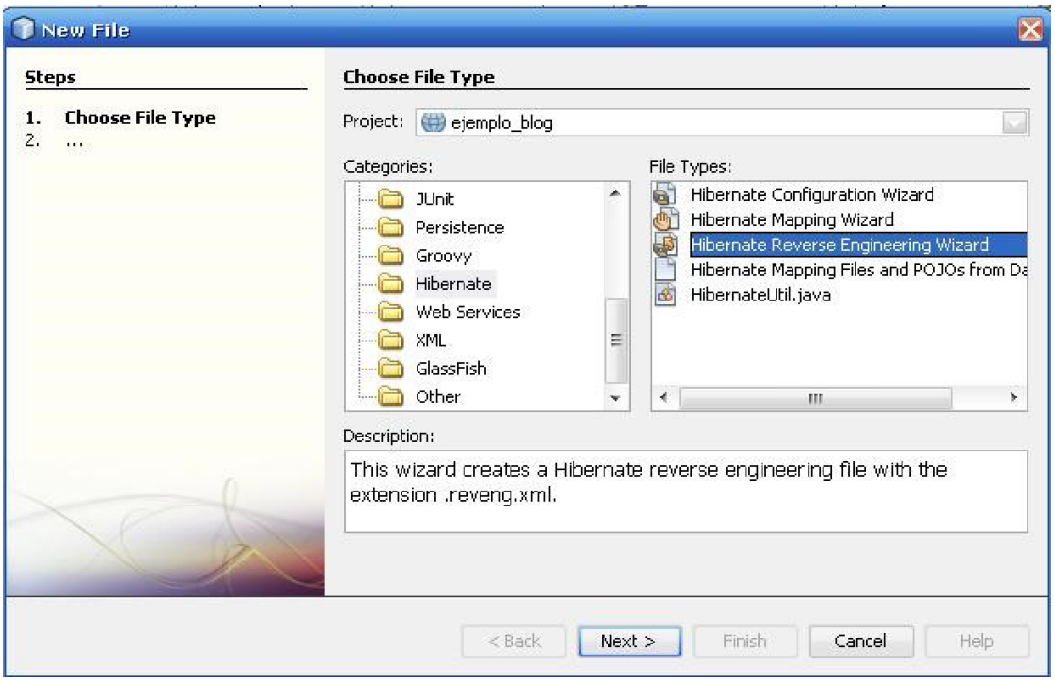
- Cuando usted busca crear archivos para múltiples tablas lo mejor es utilizar los asistentes. En este laboratorio se creará solo un POJO y un archivo de mapeo por lo que es más fácil crear los archivos individualmente

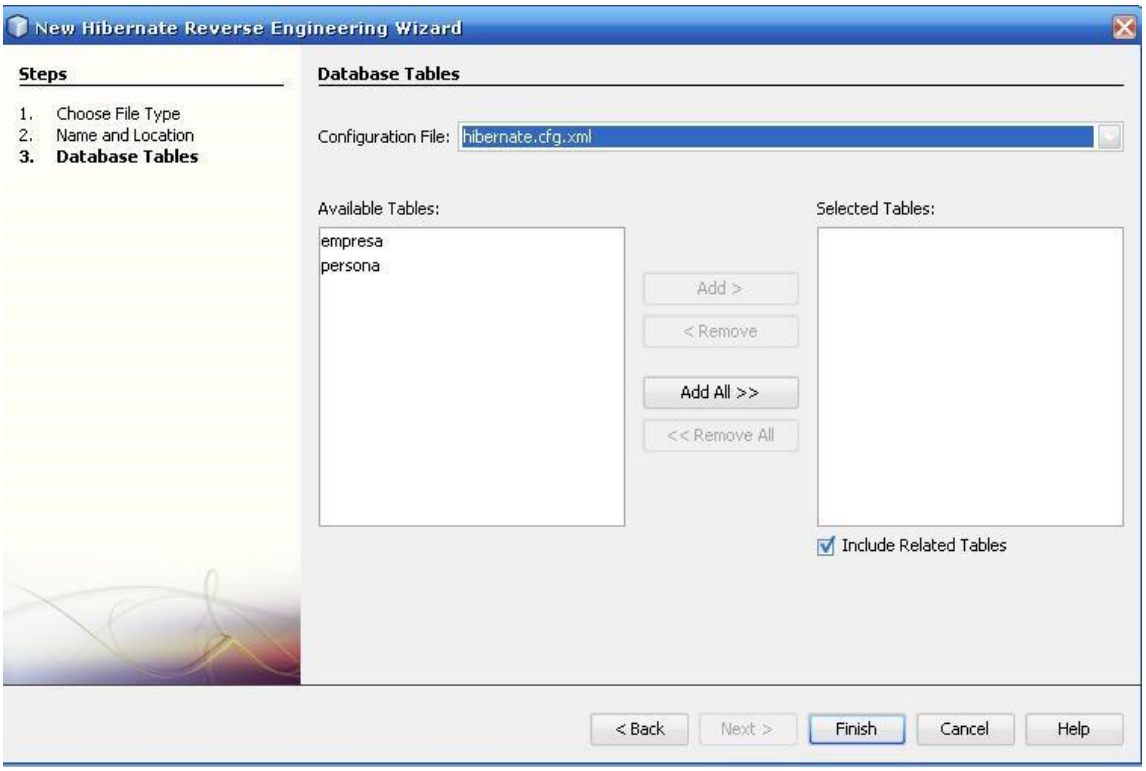
Creando un archivo de Ingeniería inversa

El archivo de ingeniería inversa (`hibernate.reveng.xml`) es un archivo en XML que puede ser utilizado para modificar los parámetros por defecto utilizados cuando se generan los archivos Hibernate desde los metadatos de la base de datos especificados en `hibernate.cfg.xml`. El asistente genera un archivo con configuraciones por defecto. Usted puede modificar el archivo para especificar explícitamente el esquema de la base de datos que es usado, para filtrar las tablas de salida que podrían no se utilizadas y para especificar como las clases de JDBC son mapeadas a clases hibernate.

1. Haga click derecho sobre el nodo Source Packages y seleccione New > Other
2. Seleccione Hibernate desde la lista de categorías y luego elija Hibernate Reverse Engineering Wizard. Haga click en Next.
3. Escriba **hibernate.reveng** para el nombre del archivo.
4. Deje por defecto la opción Location (Carpeta src). Haga click en Next.
5. Seleccione **empresa** en las tablas disponibles y haga click en Add. Luego haga click en Finish.

El asistente genera el archivo de ingeniería inversa `hibernate.reveng.xml` . Usted no necesita editar este archivo así que déjelo como esta por defecto.

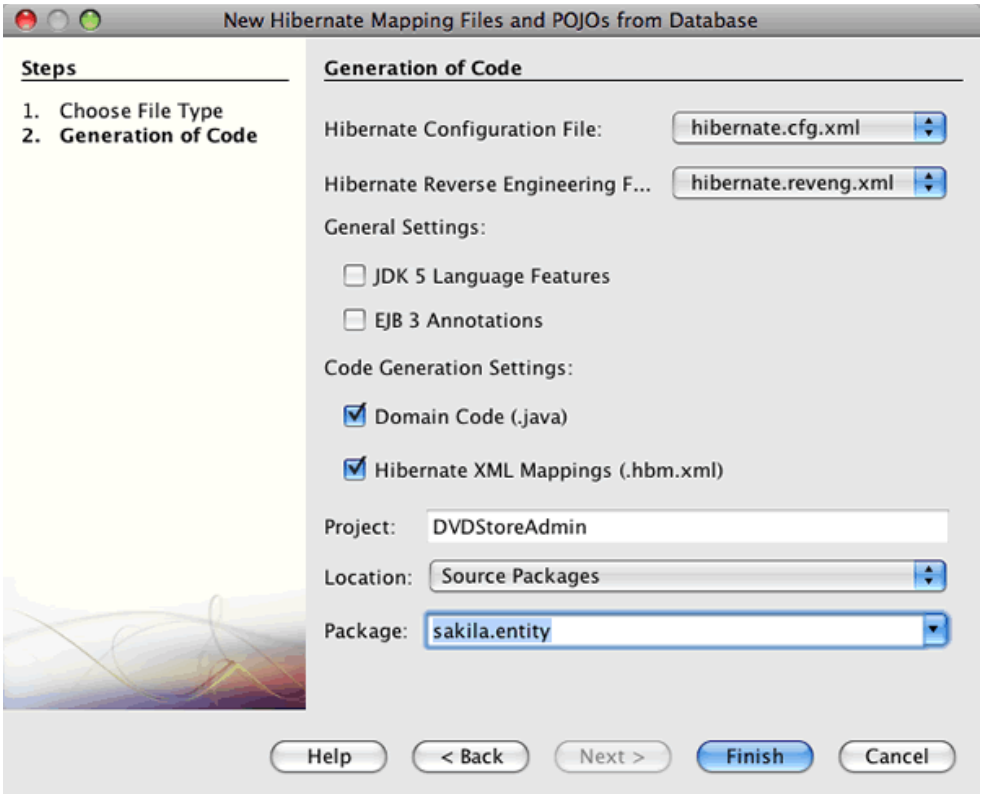

New File
Steps
1. Choose File Type
2. ...
Choose File Type
Project: ejemplo_blog
Categories:
JUnit
Persistence
Groovy
Hibernate
Web Services
XML
GlassFish
Other
File Types:
Hibernate Configuration Wizard
Hibernate Mapping Wizard
Hibernate Reverse Engineering Wizard
Hibernate Mapping Files and POJOs from Da
HibernateUtil.java
Description:
This wizard creates a Hibernate reverse engineering file with the extension .reveng.xml.
< Back
Next >
Finish
Cancel
Help


New Hibernate Reverse Engineering Wizard
Steps
1. Choose File Type
2. Name and Location
3. Database Tables
Database Tables
Configuration File: hibernate.cfg.xml
Available Tables:
empresa
persona
Add >
< Remove
Add All >>
<< Remove All
Selected Tables:
Include Related Tables
< Back
Next >
Finish
Cancel
Help

Creando Hibernate Mapping Files y POJOs desde la Base de datos

El asistente denominado Hibernate Mapping Files and POJOs from a Database generara los archivos basado en las tablas de la base de datos. Cuando usted utilice el asistente el IDE genera archivos POJOs y de mapeo basados en las tablas de la base de datos especificados en el archivo `hibernate.reveng.xml` y luego se adicionan las entradas del mapeo desde el archivo `hibernate.cfg.xml`.

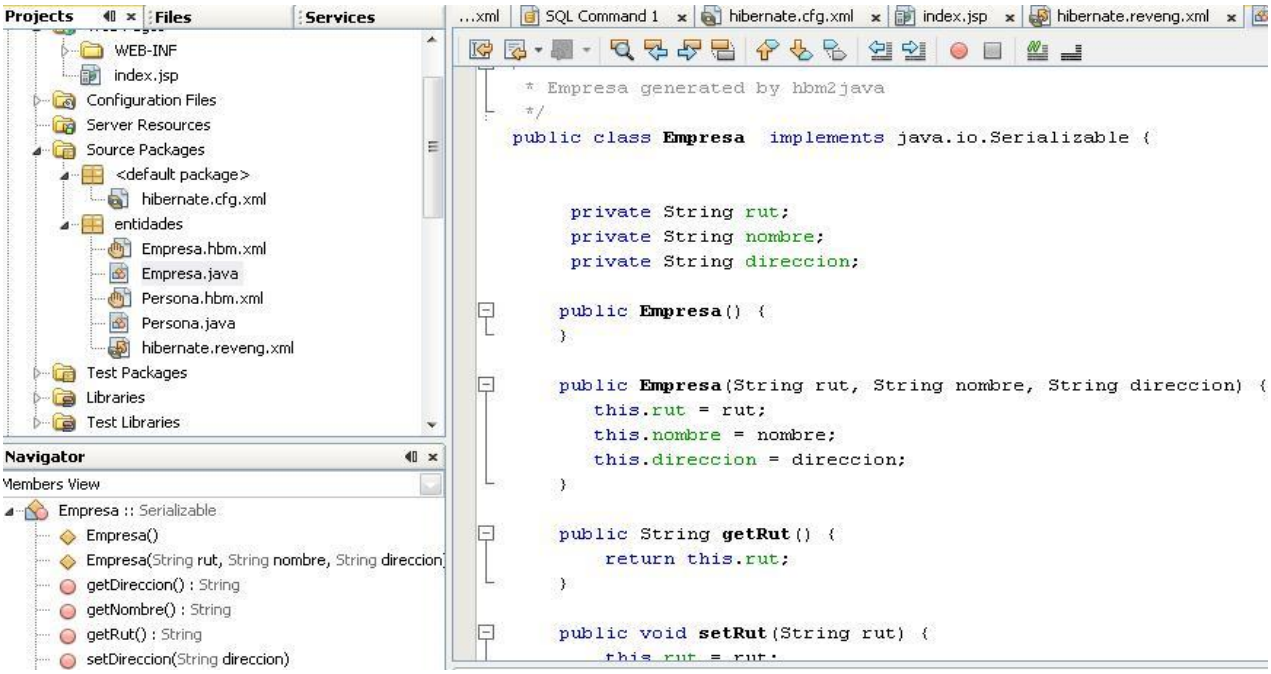
1. Haga click derecho en el nodo Source Packages y seleccione New > Other .
2. Seleccione Hibernate Mapping Files and POJOs from a Database en la categoría Hibernate. Haga click en Next.
3. Seleccione el archivo `hibernate.cfg.xml` en la sección Hibernate Configuration File si no esta seleccionado.
4. Seleccione `hibernate.reveng.xml` en la sección Hibernate Reverse Engineering File si no esta seleccionado.
5. Asegúrese que las opciones **Domain Code** y **Hibernate XML Mappings** estén seleccionados.
6. Escribe **persistencia** para el nombre del package. Haga click en el botón Finish.



Cuando haga click en Finish, el IDE genera el POJO llamado Empresa.java con todos los campos requeridos y genera el archivo de mapeo hibernate.

Ahora que tiene los archivos POJO y los relacionados de hibernate usted puede crear una sencilla interfaz GUI Java para la aplicación. También creará una consulta HQL que consulta la base de datos para recuperar los datos. En este proceso también utilizamos el editor HQL para construir y probar la consulta.

Así nos queda el proyecto

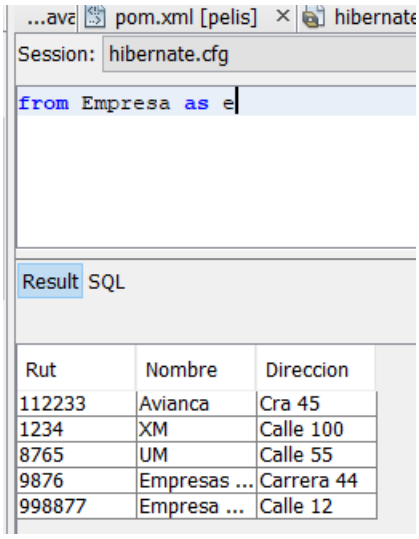


Creando una consulta en el Editor de HQL

En el IDE se puede construir y probar consultas basadas en el lenguaje de consulta de Hibernate (HQL) utilizando el Editor de consultas HQL. A medida que escribe la consulta el editor muestra el equivalente (traducido) de la consulta SQL. Al hacer click en el botón 'Run HQL Query "en la barra de herramientas, el IDE ejecuta la consulta y muestra los resultados en la parte inferior del editor.
En este ejercicio se utiliza el Editor HQL para construir consultas HQL simples que recuperan una lista de detalles de empresas sobre la base de que coincida con algunos atributos. Antes de añadir la consulta a la clase que va a utilizar, el Editor de consultas HQL probará que la conexión funciona correctamente y que la consulta produce los resultados deseados. Antes de ejecutar la consulta que primero tiene que compilar la aplicación.

1. Haga click derecho en el nodo del proyecto y seleccione Build.
2. Expanda el nodo <default package> de source package en la ventana de proyectos.
3. Haga click derecho sobre `hibernate.cfg.xml` y seleccione Run HQL Query para abrir el editor de HQL.
4. Pruebe la conexión escribiendo `from Empresa as e en el editor HQL`. Haga click en el botón Run HQL Query ( ) en la barra de herramientas..

   Cuando haga click usted vera los resultados de la consulta como se ve a continuación.



5. Escriba la siguiente consulta en el editor de consultas HQL y de click en el botón Run HQL Query, para verificar los resultados de la consulta

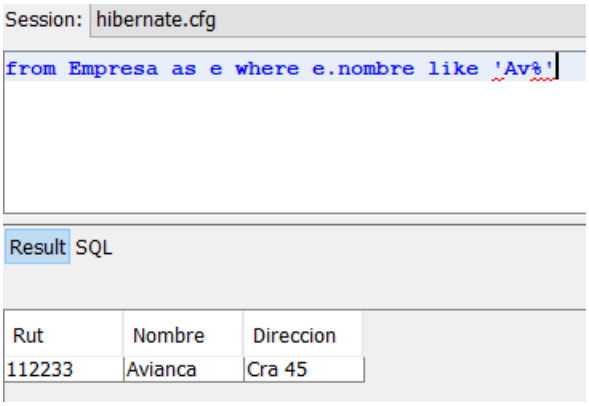
```
from Empresa
```

La consulta retorna el detalle de la lista de empresas

Si usted hace click en el boton SQL abajo de los resultados usted vera la consulta equivalente en SQL.

6. Pruebe de nuevo la siguiente consulta para buscar los nombres que inicien con 'Av'

```
from Empresa e where e.nombre like 'Av%'
```



Adicione la consulta hacia el formulario

Ahora tendrá que modificar el archivo consulta_empresa.java para añadir los métodos para construir e invocar las consultas. También es necesario modificar el controlador de eventos del botón para invocar la consulta correcta y añada un método para mostrar los resultados de la consulta en la tabla.

Abra el archivo consulta_empresa.java y haga click en el tabulador Source.

Adicione el siguiente código en la clase (en negrilla).

```
public class consulta_empresa {

    Session session = null;
    public consulta_empresa(){
       this.session=NewHibernateUtil.getSessionFactory().getCurrentSession();

    }

    public List getEmpresa(){

    List<Empresa> empre=null;

    try{
       // org.hibernate.Transaction tx =session.beginTransaction();
        session.beginTransaction();
        Query q=session.createQuery("from Empresa as empresa");
        empre= (List<Empresa>) q.list();
    }catch(Exception e){
      e.printStackTrace();
    }

    return empre;
    }
```

Los métodos llaman al método `executeHQLQuery()` y crea la consulta con una combinación de la cadena de la consulta con la cadena de búsqueda entrada por el usuario

Luego haga click derecho en el editor y elija Fix Imports (Ctrl-Shift-I) para generar la importación de los estamentos desde las librerías de Hibernate (`org.hibernate.Query`, `org.hibernate.Session`) y `java.util.List`. Guarde los cambios.

Creación de JSP

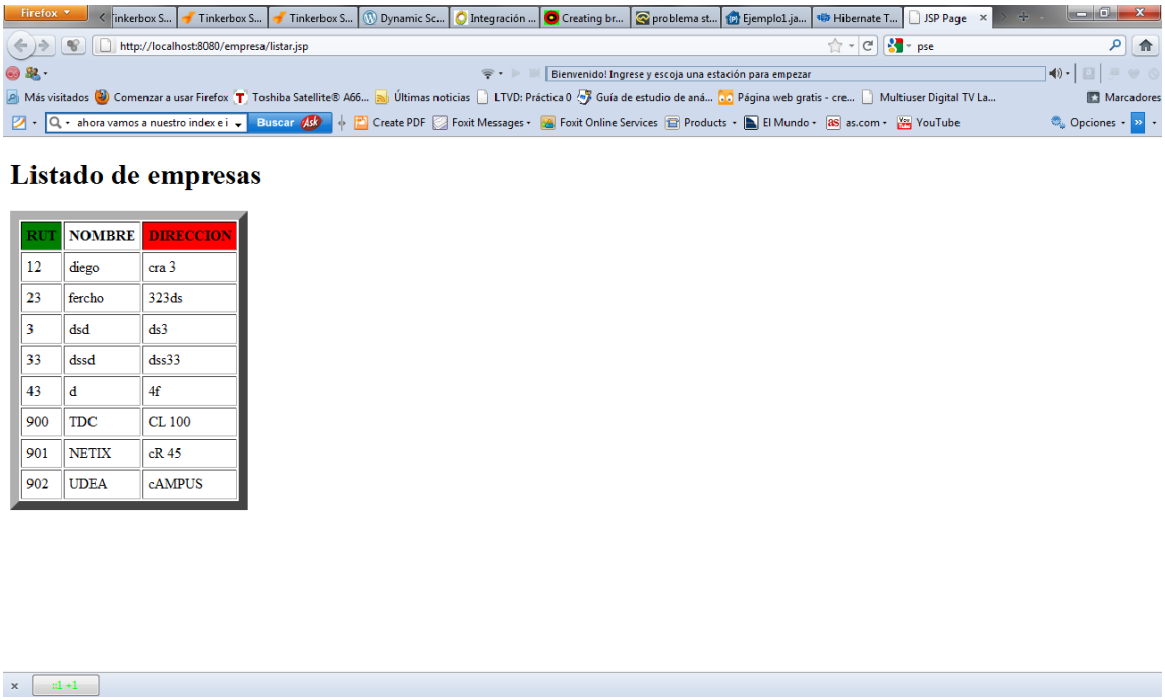
1. Crearemos las siguientes paginas JSP

- Index.jsp
- Guardar.jsp
- Respuesta.jsp
- Modificar.jsp
- Borrar.jsp

CODIGO LISTAR.JSP

```
3  <%@page import="empresa.*" %>
4  <%@page import="java.util.List" %>
5  <%@page import="java.util.Iterator" %>
6  <html>
7      <head>
8          <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
9          <title>JSP Page</title>
10     </head>
11     <body>
12         <h1>LISTADO DE EMPRESAS</h1>
13         <%
14                 consulta_empresa con=new consulta_empresa();
15                 List em=con.getEmpresa();
16                 Iterator it=em.iterator();
17                 out.println("<TABLE Border=10 CellPadding=5><TR>");
18                 out.println("<th>RUT</th><th>NOMBRE</th><th>DIRECCION</th></TR>");
19 while(it.hasNext()){
20      Empresa e=(Empresa)it.next();
21     out.println("<TR>");
22 out.println("<TD>"+e.getRut()+"</TD>");
23 out.println("<TD>"+e.getNombre()+"</TD>");
24 out.println("<TD>"+e.getDireccion()+"</TD>");
25 out.println("</TR>"); };
26 out.println("</TABLE>");       %>
27     </body>
28 </html>
```

Salida



RUT	NOMBRE	DIRECCION
12	diego	cra 3
23	fercho	323ds
3	dsd	ds3
33	dssd	dss33
43	d	4f
900	TDC	CL 100
901	NETIX	cR 45
902	UDEA	cAMPUS

Si deseamos hacer un formulario de búsqueda personalizada debemos hacer el siguiente cambio en el archivo consulta_empresa.java

```
package com.udea.logica;

import com.udea.hb.Empresa;
import com.udea.hb.NewHibernateUtil;
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;

/**
 *
 * @author Diego
 */
public class consulta_empresa {

    public List<Empresa> getEmpresa() {

        List<Empresa> empre = null;
        Session session = null;

        try {
            session = NewHibernateUtil.getSessionFactory().openSession();
            session.beginTransaction();

            Query q = session.createQuery("from Empresa");
            empre = q.list();

            session.getTransaction().commit();
```

```
        } catch (Exception e) {
            if (session != null && session.getTransaction().isActive()) {
                session.getTransaction().rollback();
            }
            e.printStackTrace();
        } finally {
            if (session != null) {
                session.close();
            }
        }

        return empre;
    }


    public List<Empresa> buscarEmpresa(String criterio, String valor) {

        List<Empresa> empre = null;
        Session session = null;

        try {
            session = NewHibernateUtil.getSessionFactory().openSession();
            session.beginTransaction();

            Query q = null;

            if (criterio.equals("rut")) {
                q = session.createQuery("from Empresa where rut = :valor");
                q.setParameter("valor", valor);
            } else if (criterio.equals("nombre")) {
                q = session.createQuery("from Empresa where nombre like :valor");
                q.setParameter("valor", "%" + valor + "%");
            }

            empre = q.list();

            session.getTransaction().commit();

        } catch (Exception e) {
            if (session != null && session.getTransaction().isActive()) {
                session.getTransaction().rollback();
            }
            e.printStackTrace();
        } finally {
            if (session != null) {
                session.close();
            }
        }

        return empre;
    }
}
```

Ahora cree el archivo buscar.jsp y coloque el siguiente código:

```
<%@page import="com.udea.hb.Empresa"%>
<%@page import="com.udea.logica.*" %>
<%@page import="java.util.List"  %>
<%@page import="java.util.Iterator"  %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Listado de Empresas</title>
</head>
<body>

<h1>LISTADO DE EMPRESAS</h1>

<!-- FORMULARIO DE BÚSQUEDA -->
<form method="post">
    Buscar por:
    <select name="criterio">
        <option value="rut">RUT</option>
        <option value="nombre">Nombre</option>
    </select>
    <input type="text" name="valor"/>
    <input type="submit" value="Buscar"/>
</form>

<br><br>

<%
    consulta_empresa con = new consulta_empresa();
    List em = null;

    String criterio = request.getParameter("criterio");
    String valor = request.getParameter("valor");

    if (valor != null && !valor.trim().equals("")) {
        em = con.buscarEmpresa(criterio, valor);
    } else {
        em = con.getEmpresa();
    }

    if (em != null) {
    Iterator it = em.iterator();

    out.println("<TABLE Border=1 CellPadding=5>");
    out.println("<TR><th>RUT</th><th>NOMBRE</th><th>DIRECCION</th></TR>");

    while(it.hasNext()) {
        Empresa e = (Empresa) it.next();
        out.println("<TR>");
        out.println("<TD>" + e.getRut() + "</TD>");
        out.println("<TD>" + e.getNombre() + "</TD>");
        out.println("<TD>" + e.getDireccion() + "</TD>");
        out.println("</TR>");
```

```
    }
    }
    out.println("</TABLE>");
%>

</body>
</html>
```

Al ejecutarlo podrá probar la búsqueda por alguno de los criterios definidos

Ejercicio:

Realice una modificación en las vistas y coloque 3 botones más para realizar el proceso de insertar, actualizar y borrar datos.

Página Recomendada: www.hibernate.org

Fecha de Entrega del Informe y Programa: 8 de Marzo 2026